Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Торосян Д.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 07.10.26

Москва, 2026

Постановка задачи

Вариант 6. Написать две отдельные программы на C. Родительский процесс получает первой строкой имя файла и открывает его на чтение. Стандартный ввод дочернего процесса перенаправляется в открытый файл, а стандартный вывод - в неименованный канал pipe1. Родитель читает результаты из канала и выводит их в свой стандартный поток вывода.

Каждая строка файла содержит произвольное количество целых чисел типа int. Дочерний процесс вычисляет сумму чисел каждой строки. Ввод и вывод выполняются системными вызовами, без stdio.

Общий метод и алгоритм решения

Использованные системные вызовы

open(filename, O_RDONLY) открывает входной файл только для чтения.

pipe(fd) создаёт канал. Из fd[0] читают данные, в fd[1] записывают.

fork() создаёт дочерний процесс. В ребёнке возвращает 0, в родителе - PID ребёнка.

dup2(oldfd, newfd) подключает стандартный ввод ребёнка к файлу, а вывод - к каналу.

execve(path, args, environ) запускает программу child в дочернем процессе.

read(fd, buffer, count) читает данные, write(fd, buffer, count) записывает их.

close(fd) закрывает ненужный файловый дескриптор.

waitpid(pid, &status, 0) ждёт завершения дочернего процесса.

_exit(code) завершает ребёнка при ошибке перенаправления или запуска программы.

Алгоритм

1. Родитель читает имя файла через read, открывает файл и создаёт канал pipe1.

2. После fork ребёнок подключает файл к stdin, а конец канала для записи - к stdout. Он закрывает лишние дескрипторы и запускает ./child через execve.

3. Дочерняя программа читает по одному символу, собирает число из цифр и учитывает его знак. На разделителе число добавляется в сумму.

4. На переводе строки ребёнок выводит сумму через write. Родитель читает байты из pipe1 и записывает их в свой stdout.

5. После окончания чтения родитель закрывает канал, вызывает waitpid и проверяет код завершения ребёнка.

Количество чисел не ограничено размером массива: входная строка целиком не хранится. Сумма имеет тип long long и должна помещаться в его диапазон. Пустые строки пропускаются, последняя строка обрабатывается и без перевода строки.

Код программы

Все файлы находятся в одном каталоге. Перед командами в Makefile стоят символы табуляции. Запуск выполняется из каталога lab1.

parent.c

```
#include <fcntl.h>
#include <sys/wait.h>
#include <unistd.h>

extern char **environ;

static int error(char *message)
{
    int length = 0;
    while (message[length] != '\0') {
        ++length;
    }
    write(STDERR_FILENO, message, length);
    return 1;
}

int main(void)
{
    char filename[1024], c;
    size_t length = 0;
    ssize_t count;

    write(STDERR_FILENO, "File: ", 6);
    while ((count = read(STDIN_FILENO, &c, 1)) > 0 && c != '\n') {
        if (length == sizeof(filename) - 1 || c == '\0') {
            return error("Invalid filename\n");
        }
        filename[length++] = c;
    }
    if (count < 0) {
        return error("Cannot read filename\n");
    }
    if (length > 0 && filename[length - 1] == '\r') {
        --length;
    }
    filename[length] = '\0';

    int file = open(filename, O_RDONLY);
    if (file < 0) {
        return error("Cannot open file\n");
    }
    int pipe1[2];
    if (pipe(pipe1) < 0) {
        return error("Cannot create pipe\n");
    }
    pid_t pid = fork();
    if (pid < 0) {
        return error("Cannot create process\n");
    }
    if (pid == 0) {
        close(pipe1[0]);
        if (dup2(file, STDIN_FILENO) < 0 || dup2(pipe1[1], STDOUT_FILENO) < 0) {
            _exit(error("Cannot redirect input/output\n"));
        }
        close(file);
        close(pipe1[1]);
        char *args[] = {"./child", NULL};
        execve("./child", args, environ);
        _exit(error("Cannot start child\n"));
    }

    close(file);
    close(pipe1[1]);
    int result = 0;
    while ((count = read(pipe1[0], &c, 1)) > 0) {
        if (write(STDOUT_FILENO, &c, 1) != 1) {
            result = error("Cannot write result\n");
            break;
        }
    }
    if (count < 0) {
        result = error("Cannot read pipe\n");
    }
    close(pipe1[0]);

    int status;
    if (waitpid(pid, &status, 0) < 0) {
        return error("Cannot wait for child\n");
    }
    return result || !WIFEXITED(status) || WEXITSTATUS(status) != 0;
}
```

child.c

```
#include <limits.h>
#include <unistd.h>

static int print_sum(long long sum)
{
    char buffer[32];
    size_t position = sizeof(buffer) - 1;
    int negative = sum < 0;
    buffer[position] = '\n';

    do {
        int digit = sum % 10;
        if (digit < 0) {
            digit = -digit;
        }
        buffer[--position] = '0' + digit;
        sum /= 10;
    } while (sum != 0);
    if (negative) {
        buffer[--position] = '-';
    }

    for (; position < sizeof(buffer); ++position) {
        if (write(STDOUT_FILENO, &buffer[position], 1) != 1) {
            return 1;
        }
    }
    return 0;
}

int main(void)
{
    long long number = 0, sum = 0;
    int sign = 1, has_number = 0, has_line = 0;
    char c;

    for (;;) {
        ssize_t count = read(STDIN_FILENO, &c, 1);
        if (count < 0) {
            return 1;
        }

        if (count == 0) {
            c = '\n';
        }
        if (c >= '0' && c <= '9') {
            number = number * 10 + (c - '0');
            if (number > (sign > 0 ? INT_MAX : -(long long)INT_MIN)) {
                return 1;
            }
            has_number = 1;
        } else if (c == '-' || c == '+') {
            if (has_number) {
                return 1;
            }
            sign = c == '-' ? -1 : 1;
        } else if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
            if (has_number) {
                sum += sign * number;
                has_line = 1;
            }
            number = 0;
            sign = 1;
            has_number = 0;
            if (c == '\n' && has_line) {
                if (print_sum(sum) != 0) {
                    return 1;
                }
                sum = 0;
                has_line = 0;
            }
        } else {
            return 1;
        }
        if (count == 0) {
            break;
        }
    }
    return 0;
}
```

Makefile

```
all:
    cc parent.c -o parent
    cc child.c -o child

clean:
    rm -f parent child
```

Протокол работы программы

Примеры запуска из каталога с исходными файлами. Команда printf оболочки создаёт тестовые файлы и передаёт их имена родительской программе. Приглашение File: выводится в stderr.

Тест 1. Несколько строк входного файла

```
$ cat input.txt
1 2 3
-10 5 2
0
+7 -4 20 0
2147483647 -2147483648
$ make
cc parent.c -o parent
cc child.c -o child
$ printf 'input.txt\n' | ./parent
File: 6
-3
0
23
-1
```

Тест 2. Отрицательные числа и сумма за пределами int

```
$ printf '%s\n' '-12 7 0' '2147483647 2147483647' > test2.txt
$ printf 'test2.txt\n' | ./parent
File: -5
4294967294
```

Тест 3. Пустая строка и ввод без перевода строки

```
$ printf '\n1 2 3' > test3.txt
$ printf 'test3.txt\n' | ./parent
File: 6
```

Тест 4. Произвольное количество чисел

```
$ printf '1 %.0s' {1..1000} > test4.txt
$ printf '\n' >> test4.txt
$ printf 'test4.txt\n' | ./parent
File: 1000
```

В последнем тесте одна строка содержит 1000 единиц, разделённых пробелами. Результат равен 1000. Во втором тесте сумма двух чисел INT_MAX равна 4294967294 и корректно хранится в long long.

Вывод

Реализованы две отдельные программы, взаимодействующие через неименованный канал. Родитель открывает файл, перенаправляет потоки ребёнка и выводит полученные результаты. Ребёнок разбирает целые числа и вычисляет суммы строк. Проверены несколько строк, отрицательные числа, ноль, сумма за пределами int, пустая строка, ввод без перевода строки и строка из 1000 чисел. Сборка выполняется командой make, ввод и вывод - через read и write.